



fundación
universitaria

A young man with brown hair, wearing a blue denim shirt over a white t-shirt, is leaning over a desk, pointing at a laptop screen. A young woman with long dark hair, wearing a white blazer, is sitting at the desk, looking at the laptop with a slight smile. The background is a solid orange color. The image is partially obscured by a white diagonal shape on the left side.

Introducción

RodApp es una plataforma digital de gestión integral para motociclistas, diseñada para cubrir de forma centralizada todas las necesidades administrativas, preventivas y operativas del propietario de una motocicleta. El proyecto surge de la observación de que los conductores no cuentan con una herramienta unificada que les permita hacer seguimiento de sus documentos legales, registros de combustible, historial de mantenimiento y recorridos, todo desde un solo lugar.

El presente documento recoge la definición formal del problema, los objetivos del sistema, el alcance funcional, los requerimientos de software, la propuesta de diagramas de diseño y la arquitectura del back-end planificada. Este documento está dirigido a los interesados técnicos y de negocio del proyecto y representa el estado actual del desarrollo: front-end web completamente construido, con arquitectura back-end en fase de planificación.

La plataforma opera en dos frentes complementarios: una aplicación web progresiva (PWA) desarrollada con HTML5, CSS3 y JavaScript vanilla bajo paradigma mobile-first, el cual se describe en detalle en la sección 5 de este documento.

Tabla de Contenido

Introducción	2
Tabla de Contenido	3
1. Definición del Problema a Solucionar	5
1.1 Problemáticas Identificadas	5
1.2 Contexto del Proyecto	5
2. Objetivos	6
2.1 Objetivo General	6
2.2 Objetivos Específicos	6
3. Alcance y Delimitaciones	7
3.1 Alcance Funcional	7
3.2 Funcionalidades Excluidas del Alcance Actual	8
3.3 Delimitaciones Técnicas	8
4. Diseño de Software	9
4.1 Requerimientos	9
4.1.1 Requerimientos Funcionales	9
4.1.2 Requerimientos No Funcionales	13
4.2 Casos de Uso	15
Actores del Sistema	15
Casos de Uso Principales	15
4.3 Diagrama de Secuencia	16
Flujos Priorizados para Diagramar	16
4.4 Diagrama de Clases	17
Clases del Dominio (Backend y Móvil)	17
4.5 Modelo Entidad-Relación	18
Entidades y Relaciones Propuestas	18
Relaciones Principales del MER	19
5. Arquitectura Back-End	20
5.1 Ficha Técnica del Backend	20
5.2 Arquitectura de Capas	21
Responsabilidades por Capa	21

5.3 Estructura de Directorios del Proyecto Backend	22
5.4 Endpoints REST Principales	23
5.5 Ciclo de Vida de una Petición HTTP	24
5.6 Módulo de Viajes y Mapa Web.....	¡Error! Marcador no definido.
5.7 Seguridad y Autenticación	24
5.8 Patrones de Diseño Aplicados	25
5.9 Stack de Dependencias Maven	25
5.10 Consideraciones para la Fase de Desarrollo	26

1. Definición del Problema a Solucionar

Los propietarios de motocicletas en Colombia enfrentan un problema sistemático de desorganización en la gestión de su vehículo. Actualmente, la información relevante de una moto —mantenimientos, documentación legal, registros de combustible y recorridos— se distribuye entre notas físicas, aplicaciones genéricas no especializadas y la memoria del conductor. Esta dispersión genera consecuencias concretas en la seguridad vial y la economía del usuario.

1.1 Problemáticas Identificadas

- Pérdida de trazabilidad histórica: sin un registro organizado, el propietario no puede revisar el historial de mantenimientos ni calcular consumos reales con precisión.
- Vencimiento de documentos obligatorios: el SOAT y la RTM tienen fechas de vencimiento que pueden generar infracciones de tránsito y riesgos legales.
- Mantenimientos preventivos postergados: sin un sistema de alertas basado en kilometraje o tiempo, los cambios de aceite, frenos, cadena y otros consumibles se realizan de forma reactiva y tardía.
- Ausencia de control de gastos: la falta de registro sistemático de combustible y servicios impide al conductor evaluar el costo real de operación de su motocicleta.
- Fragmentación de herramientas: las soluciones existentes en el mercado como Drivvo o Fuelio son genéricas, no están adaptadas al contexto colombiano y no integran todos los módulos en una sola experiencia.

1.2 Contexto del Proyecto

RodApp nace como respuesta a esta problemática, ofreciendo una plataforma unificada disponible tanto en web como en dispositivo móvil Android. El sistema está contextualizado para el entorno colombiano, contemplando los documentos y regulaciones locales (SOAT, RTM, licencia de conducción). La plataforma web ha sido desarrollada bajo un paradigma mobile-first con soporte responsive completo, y la arquitectura backend está diseñada para servir de forma simultánea a ambos clientes (web y móvil).

2. Objetivos

2.1 Objetivo General

Desarrollar una plataforma digital multiplataforma (web y móvil Android) que centralice el registro, seguimiento y análisis de la información operativa, preventiva y legal de motocicletas, proporcionando herramientas de alerta inteligente, historial de recorridos y gestión de documentos para el motociclista colombiano.

2.2 Objetivos Específicos

1. Implementar un módulo de autenticación de usuarios con soporte para inicio de sesión con Google OAuth2, que permita acceso seguro tanto desde la plataforma web como desde la aplicación móvil.
2. Desarrollar un sistema de registro y gestión de motocicletas que permita almacenar los datos técnicos del vehículo, su fotografía y los documentos legales asociados (SOAT, RTM, licencia de conducción y documentos personalizados).
3. Construir un módulo de registro de combustible con cálculo automático del consumo promedio por kilómetro y el costo operativo acumulado por tanqueada.
4. Implementar un sistema de mantenimiento preventivo que genere alertas automáticas basadas en intervalos de kilometraje y tiempo para los consumibles mecánicos del vehículo.
5. Integrar un módulo de mapas en la plataforma web que muestre el historial de viajes del usuario con la trayectoria georreferenciada de cada recorrido registrado.
6. Diseñar y construir el backend REST bajo una arquitectura de tres capas (Controller, Service, DAO) con Jakarta EE, que sirva de forma simultánea a los clientes web y móvil.
7. Proveer un sistema de notificaciones que informe oportunamente al usuario sobre vencimientos de documentos y mantenimientos programados.
8. Generar reportes y estadísticas de uso, consumo y gastos que permitan al usuario tomar decisiones informadas sobre el mantenimiento y operación de su motocicleta.

3. Alcance y Delimitaciones

3.1 Alcance Funcional

RodApp abarca el ciclo completo de gestión de una motocicleta desde el punto de vista del propietario. Las funcionalidades incluidas en el alcance actual del proyecto son:

PLATAFORMA WEB

- Pantalla de login con autenticación vía correo/contraseña y Google Sign-In.
- Dashboard principal con resumen de estado del vehículo, documentos próximos a vencer y acciones rápidas.
- Módulo de garaje: registro multi-moto con datos técnicos, fotografía y estado documental por vehículo.
- Registro de motocicleta en tres pasos: datos básicos, datos técnicos y confirmación.
- Gestión de documentos: SOAT, RTM, licencia de conducción y documentos personalizados con alertas de vencimiento.
- Mantenimiento predictivo: odómetro hero, alertas por criticidad (urgente/advertencia/informativo) y registro histórico de servicios.
- Módulo de mapa: visualización del historial de viajes con trayecto georreferenciado por recorrido (integración con Leaflet.js / Google Maps API / Mapbox).
- Historial de viajes: estadísticas de distancia, duración, consumo y timeline interactivo.
- Registro de combustible: formulario de carga con cálculo automático de consumo promedio y costo operativo.
- Centro de notificaciones: alertas categorizadas por tipo con marcado de leído/no leído.
- Perfil de usuario: configuración personal, preferencias de sistema y opciones de seguridad.
- Sección de tips: base de conocimiento de consejos para el motociclista.

3.2 Funcionalidades Excluidas del Alcance Actual

- Marketplace de repuestos o servicios mecánicos de terceros.
- Funcionalidades de comunidad (foro, compartir rutas entre usuarios).
- Integración con dispositivos IoT o diagnóstico OBD-II.
- Machine learning o predicción de fallos.
- Exportación de reportes en formato PDF (considerada para fases posteriores).

3.3 Delimitaciones Técnicas

Aspecto	Delimitación
Plataforma web	HTML5 + CSS3 + JavaScript vanilla. Mobile-first responsive. Soporta móvil (430px), tablet (768px) y desktop (1024px+).
Plataforma móvil	Android nativo con Kotlin y arquitectura MVVM. Versiones modernas de Android.
Backend	Jakarta EE 10 + Jersey JAX-RS + Hibernate JPA + MySQL. Desplegado en Tomcat.
Almacenamiento móvil	Room Database para persistencia local offline-first.
Mapa (web)	Historial de viajes georreferenciados. No incluye navegación en tiempo real.
Autenticación	JWT + Google OAuth2. Sin soporte para Apple Sign-In ni otros proveedores.
Contexto geográfico	Colombia: SOAT, RTM y regulaciones de tránsito colombianas.

4. Diseño de Software

4.1 Requerimientos

4.1.1 Requerimientos Funcionales

Los siguientes requerimientos funcionales cubren tanto la plataforma web.

RF — AUTENTICACIÓN Y PERFIL

ID	Descripción	Prioridad
RF-01	El sistema debe permitir el registro de nuevos usuarios mediante correo electrónico y contraseña.	Alta
RF-02	El sistema debe permitir al usuario registrar y editar su nombre, foto de perfil y datos personales.	Alta
RF-03	El sistema debe gestionar sesiones de usuario con la base de datos	Alta

RF — GESTIÓN DE MOTOCICLETA

ID	Descripción	Prioridad
RF-04	El sistema debe permitir registrar una o más motocicletas asociadas al usuario, con marca, modelo, año, cilindrada, color, tipo de combustible y fotografía.	Alta
RF-05	El sistema debe permitir registrar el kilometraje inicial de la motocicleta al momento del registro.	Alta
RF-06	El sistema debe permitir visualizar, editar y eliminar los datos técnicos de cada motocicleta registrada.	Alta
RF-07	El sistema debe mostrar el estado general de cada moto (documentos, mantenimientos pendientes) en el módulo de garaje.	Alta

RF — GESTIÓN DE DOCUMENTOS

ID	Descripción	Prioridad
RF-08	El sistema debe permitir registrar el SOAT con número de póliza, expedidor, fecha de emisión y fecha de vencimiento.	Alta
RF-09	El sistema debe permitir registrar la RTM con número de certificado, fecha de realización y fecha de vencimiento.	Alta
RF-10	El sistema debe permitir registrar la licencia de conducción con número, categoría y fecha de vencimiento.	Alta
RF-11	El sistema debe permitir crear documentos personalizados con nombre, número, fechas y archivo adjunto.	Media
RF-12	El sistema debe clasificar cada documento por su estado: Vigente (verde), Próximo a vencer (naranja) o Vencido (rojo), según la fecha actual.	Alta
RF-13	El sistema debe enviar notificaciones preventivas cuando un documento esté a 30 o menos días de vencer.	Alta

RF — COMBUSTIBLE

ID	Descripción	Prioridad
RF-14	El sistema debe permitir registrar eventos de tanqueada con: fecha, tipo de combustible, cantidad (galones/litros), costo total y lectura del odómetro.	Alta
RF-15	El sistema debe calcular automáticamente el rendimiento de combustible (km/galón o km/litro) a partir de registros consecutivos.	Alta
RF-16	El sistema debe mostrar el costo acumulado de combustible por periodo (semana, mes, año).	Media
RF-17	El sistema debe listar el historial de tanqueadas ordenadas por fecha.	Alta

RF — MANTENIMIENTO

ID	Descripción	Prioridad
RF-18	El sistema debe permitir registrar eventos de mantenimiento con: tipo de servicio, fecha, kilometraje, costo, técnico responsable y observaciones.	Alta
RF-19	El sistema debe permitir definir intervalos de mantenimiento (por km o por tiempo) para cada tipo de consumible.	Alta
RF-20	El sistema debe calcular y mostrar cuántos kilómetros o días faltan para el próximo mantenimiento de cada consumible.	Alta

RF-21	El sistema debe generar alertas cuando un mantenimiento esté próximo (nivel informativo, advertencia o urgente) según la proximidad al intervalo definido.	Alta
RF-22	El sistema debe mostrar el historial de mantenimientos organizados por mes y tipo de servicio.	Media

RF — NOTIFICACIONES

ID	Descripción	Prioridad
RF-23	El sistema debe enviar notificaciones al usuario sobre vencimientos de documentos (SOAT, RTM, licencia).	Alta
RF-24	El sistema debe enviar notificaciones sobre mantenimientos próximos según los intervalos definidos.	Alta
RF-25	El sistema debe permitir al usuario marcar notificaciones como leídas y filtrarlas por categoría.	Media

4.1.2 Requerimientos No Funcionales

ID	Categoría	Descripción
RNF-01	Usabilidad	La interfaz debe ser intuitiva, operable sin conocimientos técnicos y seguir los principios de Material Design. La navegación principal debe ser accesible desde cualquier pantalla de la aplicación.
RNF-02	Rendimiento	El sistema debe responder a las solicitudes del usuario en menos de 2 segundos bajo condiciones normales de red. Los endpoints REST deben procesar solicitudes en menos de 500ms.
RNF-03	Disponibilidad offline	La aplicación móvil debe funcionar sin conexión a internet, almacenando datos localmente en Room Database y sincronizando cuando haya conectividad.
RNF-04	Persistencia	Los datos deben persistir correctamente tanto en el backend (MySQL) como localmente en el dispositivo móvil, sin pérdida de información ante cierres inesperados.
RNF-05	Seguridad	Todos los endpoints REST deben requerir autenticación JWT. Las contraseñas deben almacenarse con hashbcrypt. Los datos personales deben cumplir con la Ley 1581 de 2012 (Habeas Data, Colombia).
RNF-06	Escalabilidad	La arquitectura de tres capas debe permitir agregar nuevos módulos o entidades sin modificar las capas existentes.
RNF-07	Compatibilidad	La plataforma web debe funcionar en Chrome, Firefox, Safari y Edge modernos. La app móvil debe ser compatible con versiones recientes de Android.

RNF-08	Responsividad	La plataforma web debe adaptarse correctamente a pantallas de móvil ($\leq 430\text{px}$), tablet (768px) y desktop ($\geq 1024\text{px}$).
RNF-09	Mantenibilidad	El código debe seguir la arquitectura MVC-DAO en el backend, con separación clara de responsabilidades y documentación de endpoints con OpenAPI/Swagger.

4.2 Casos de Uso

Actores del Sistema

- Usuario Motociclista: actor principal que interactúa con todas las funcionalidades de la plataforma (registro de datos, consulta de historial, configuración de alertas).
- Sistema de Notificaciones: actor secundario que ejecuta el envío de alertas de forma automática según eventos del sistema.
- Servicio de Mapas (API Externa): actor externo que provee la visualización cartográfica para el módulo de historial de viajes.

Casos de Uso Principales

CU	Nombre	Actor Principal
CU-01	Registrar cuenta de usuario	Usuario Motociclista
CU-02	Iniciar sesión (email/Google)	Usuario Motociclista / Google OAuth2
CU-03	Registrar motocicleta	Usuario Motociclista
CU-04	Gestionar documentos legales	Usuario Motociclista
CU-05	Registrar tanqueada	Usuario Motociclista
CU-06	Consultar estadísticas de combustible	Usuario Motociclista
CU-07	Registrar mantenimiento	Usuario Motociclista
CU-08	Configurar intervalos de mantenimiento	Usuario Motociclista
CU-09	Ver historial de viajes en el mapa	Usuario Motociclista / API de Mapas
CU-10	Recibir notificación de vencimiento	Sistema de Notificaciones
CU-11	Gestionar perfil de usuario	Usuario Motociclista
CU-12	Consultar tips y consejos	Usuario Motociclista

4.3 Diagrama de Secuencia

Se plantea la elaboración de diagramas de secuencia para los flujos más representativos del sistema. Cada diagrama mostrará la interacción temporal entre los componentes: Cliente, API REST (Controller), Service, DAO y Base de Datos.

Flujos Priorizados para Diagramar

- Secuencia 1 — Registro de tanqueada: validación de campos en el cliente, envío al endpoint REST, cálculo automático del rendimiento en la capa Service, persistencia en DAO y retorno de los cálculos al cliente.
- Secuencia 2 — Consulta de historial de viajes en el mapa: solicitud al endpoint de viajes, recuperación de coordenadas GPS desde la base de datos, formateo como GeoJSON y renderización en el cliente con la API de mapas.
- Secuencia 3 — Generación de alerta de vencimiento de documento: job programado que consulta documentos, evalúa fechas de vencimiento, genera notificación en base de datos y la empuja al cliente.
- Secuencia 4 — Registro de mantenimiento y actualización de próximo servicio: guardado del evento, recálculo del próximo intervalo basado en $\text{km actual} + \text{intervalo}$ configurado y actualización del estado de alerta.

4.4 Diagrama de Clases

El diagrama de clases representa la estructura estática del sistema, incluyendo las entidades principales del dominio, sus atributos, métodos y las relaciones entre ellas. Se plantea su elaboración en dos contextos: el modelo de dominio del backend.

Clases del Dominio

Clase	Atributos Principales	Relaciones
Usuario	id, nombre, email, passwordHash, fotoPerfil, fechaRegistro	1 Usuario → N Motocicletas
Motocicleta	id, marca, modelo, año, placa, cilindrada, tipo Fuel, color, kmActual, foto	1 Moto → N Documentos, N Mantenimientos, N Tanqueadas, N Viajes
Documento	id, tipo, numero, expedidor, fechaEmision, fechaVencimiento, archivo	N Documentos → 1 Moto
Tanqueada	id, fecha, tipoFuel, cantidad, costo, odometro, ubicacion, observaciones	N Tanqueadas → 1 Moto
Mantenimiento	id, tipo, fecha, kilometraje, costo, tecnico, descripcion, proximoKm	N Mantenimientos → 1 Moto
Notificación	id, tipo, titulo, mensaje, leida, fechaCreacion	N Notificaciones → 1 Usuario
IntervaloPrev.	id, tipoServicio, intervalKm, intervalDias, descripcion	N Intervalos → 1 Moto

4.5 Modelo Entidad–Relación

El Modelo Entidad–Relación (MER) define la estructura de la base de datos relacional MySQL que soporta el backend de RodApp. A continuación se describe la propuesta de tablas, claves primarias, claves foráneas y relaciones:

Entidades y Relaciones Propuestas

- usuarios (id PK, nombre, email UNIQUE, password_hash, foto_perfil, fecha_registro, activo)
- motocicletas (id PK, usuario_id FK→usuarios, marca, modelo, año, placa UNIQUE, cilindrada, tipo_combustible, color, km_actual, foto, fecha_registro)
- documentos (id PK, moto_id FK→motocicletas, tipo ENUM[SOAT,RTM,LICENCIA,PERSONALIZADO], numero, expedidor, fecha_emision, fecha_vencimiento, ruta_archivo)
- tanqueadas (id PK, moto_id FK→motocicletas, fecha, tipo_combustible, cantidad, costo, odometro, latitud, longitud, observaciones, rendimiento_calculado)
- mantenimientos (id PK, moto_id FK→motocicletas, tipo_servicio, fecha, kilometraje, costo, tecnico, descripcion, proximo_km, proximo_fecha)
- notificaciones (id PK, usuario_id FK→usuarios, tipo ENUM[DOCUMENTO,MANTENIMIENTO,SISTEMA], titulo, mensaje, leida BOOL, fecha_creacion)
- intervalos_preventivos (id PK, moto_id FK→motocicletas, tipo_servicio, intervalo_km, intervalo_dias, descripcion)

Relaciones Principales del MER

Relación	Cardinalidad	Tipo	Descripción
usuarios → motocicletas	1 : N	Has Many	Un usuario puede tener varias motos registradas
motocicletas → documentos	1 : N	Has Many	Cada moto puede tener múltiples documentos legales
motocicletas → tanqueadas	1 : N	Has Many	Cada moto acumula el historial de tanqueadas
motocicletas → mantenimientos	1 : N	Has Many	Cada moto lleva su historial de mantenimientos
usuarios → notificaciones	1 : N	Has Many	Cada usuario recibe notificaciones personalizadas
motocicletas → intervalos	1 : N	Has Many	Cada moto puede tener intervalos configurados por consumible

5. Arquitectura Back-End

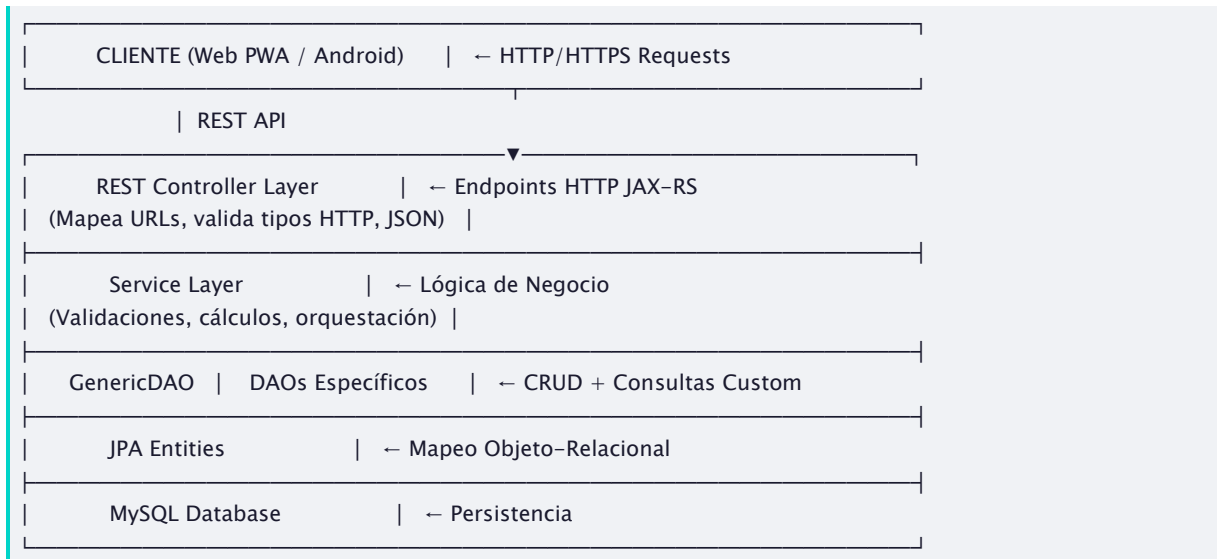
El backend de RodApp está diseñado bajo una arquitectura de tres capas (Controller → Service → DAO), implementada con Jakarta EE 10 y probada en el proyecto académico GestionInventarios como base de referencia. Esta arquitectura está adaptada para servir simultáneamente a la plataforma web (PWA) y a la aplicación móvil nativa Android, actuando como API REST centralizada.

5.1 Ficha Técnica del Backend

Aspecto	Valor
Nombre	RodApp REST API — Arquitectura MVC-DAO
Tipo de aplicación	Web Application REST API (WAR)
Java Versión	17+
Framework Web	Jakarta EE 10 + Jersey 3.1.1 (JAX-RS)
ORM	Hibernate JPA 6.2.0
Base de Datos	MySQL 8.0+
Build Tool	Maven
Servidor de despliegue	Apache Tomcat 10+
Patrón Principal	3 Capas (Controller - Service - DAO)
Autenticación	JWT (JSON Web Token)
Clientes Soportados	Plataforma Web (PWA)
Formato de Datos	JSON (serializado por Jersey/Jackson)

5.2 Arquitectura de Capas

La arquitectura sigue el patrón de tres capas con responsabilidades claramente delimitadas. El flujo de datos siempre recorre las capas de forma unidireccional: el cliente hace una solicitud HTTP que llega al Controller, que delega en el Service, que a su vez utiliza el DAO para interactuar con la base de datos:



Responsabilidades por Capa

Capa	Responsabilidades	Restricciones
Controller	Mapear URLs a métodos HTTP, validar tipos de datos HTTP, llamar al Service, serializar respuestas JSON.	No realiza validaciones de negocio. No accede a la BD directamente.
Service	Validar reglas de negocio, ejecutar cálculos (rendimiento, intervalos), orquestar múltiples DAOs, manejar excepciones.	No construye respuestas HTTP. No escribe SQL directo.
DAO	Operaciones CRUD sobre entidades JPA, consultas especializadas (HQL/JPQL), gestión de transacciones ACID y EntityManager.	No valida lógica de negocio. No maneja HTTP ni REST.

5.3 Estructura de Directorios del Proyecto Backend

```
rodapp-backend/
├── src/main/java/com/rodapp/
│   ├── controller/
│   │   ├── AuthController.java      ← Login, registro, Google OAuth2
│   │   ├── UsuarioController.java
│   │   ├── MotoController.java
│   │   ├── DocumentoController.java
│   │   ├── TanqueadaController.java
│   │   ├── MantenimientoController.java
│   │   ├── ViajeController.java     ← Historial de viajes (mapa web)
│   │   ├── NotificacionController.java
│   │   └── TipController.java
│   ├── service/
│   │   ├── AuthService.java
│   │   ├── UsuarioService.java
│   │   ├── MotoService.java
│   │   ├── DocumentoService.java    ← Calcula estado: Vigente/Próximo/Vencido
│   │   ├── TanqueadaService.java    ← Calcula rendimiento km/l automáticamente
│   │   ├── MantenimientoService.java ← Calcula próximo servicio por km/fecha
│   │   ├── ViajeService.java        ← Procesa y entrega coordenadas GeoJSON
│   │   └── NotificacionService.java  ← Job de alertas automáticas
│   ├── dao/
│   │   ├── GenericDAO.java          ← Plantilla CRUD reutilizable
│   │   ├── UsuarioDAO.java
│   │   ├── MotoDAO.java
│   │   ├── DocumentoDAO.java
│   │   ├── TanqueadaDAO.java
│   │   ├── MantenimientoDAO.java
│   │   ├── ViajeDAO.java
│   │   └── NotificacionDAO.java
│   ├── model/entity/
│   │   ├── Usuario.java
│   │   ├── Motocicleta.java
│   │   ├── Documento.java
│   │   ├── Tanqueada.java
│   │   ├── Mantenimiento.java
│   │   ├── Viaje.java
│   │   └── Notificacion.java
│   ├── security/
│   │   ├── JwtUtil.java              ← Generación y validación de tokens JWT
│   │   ├── JwtFilter.java            ← Filtro de autenticación por Request
│   │   └── GoogleOAuthService.java   ← Validación token Google OAuth2
│   └── util/
│       └── JpaUtil.java              ← EntityManagerFactory Singleton
├── src/main/resources/META-INF/
├── persistence.xml                  ← Configuración JPA/Hibernate/MySQL
└── pom.xml                          ← Dependencias Maven
```

5.4 Endpoints REST Principales

La API REST de RodApp expone los siguientes grupos de endpoints. Todos los endpoints (excepto /auth/*) requieren Bearer Token JWT en el header de autorización:

Método	Endpoint	Descripción	Auth
POST	/auth/register	Registro de nuevo usuario (email + password)	Pública
POST	/auth/login	Login con email/password → retorna JWT	Pública
GET	/usuarios/me	Obtener perfil del usuario autenticado	JWT
PUT	/usuarios/me	Actualizar perfil del usuario	JWT
GET	/motos	Listar motocicletas del usuario	JWT
POST	/motos	Registrar nueva motocicleta	JWT
PUT	/motos/{id}	Actualizar datos de la moto	JWT
DELETE	/motos/{id}	Eliminar moto y datos asociados	JWT
GET	/motos/{id}/documentos	Listar documentos de una moto	JWT
POST	/motos/{id}/documentos	Registrar nuevo documento	JWT
PUT	/documentos/{id}	Actualizar documento	JWT
GET	/motos/{id}/tanqueadas	Historial de tanqueadas de la moto	JWT
POST	/motos/{id}/tanqueadas	Registrar tanqueada (calcula rendimiento automáticamente)	JWT
GET	/motos/{id}/mantenimientos	Historial de mantenimientos	JWT
POST	/motos/{id}/mantenimientos	Registrar evento de mantenimiento	JWT
GET	/notificaciones	Listar notificaciones del usuario	JWT
PUT	/notificaciones/{id}/leer	Marcar notificación como leída	JWT

5.5 Ciclo de Vida de una Petición HTTP

El siguiente ejemplo ilustra el flujo completo de una petición para registrar una tanqueada, que incluye cálculo automático de rendimiento:

```
1. ANDROID/WEB → POST /motos/3/tanqueadas (Bearer JWT + datos de la carga)
2. JwtFilter → Valida el token JWT → extrae usuario_id
3. TanqueadaController → Deserializa JSON → llama TanqueadaService.registrar()
4. TanqueadaService → Valida reglas de negocio (odómetro > último registro)
5. TanqueadaService → Consulta última tanqueada vía TanqueadaDAO.findLastByMoto()
6. TanqueadaService → Calcula rendimiento = (km_actual - km_anterior) / litros
7. TanqueadaDAO → Begin Transaction → persist(nuevaTanqueada) → Commit
8. JPA/Hibernate → INSERT INTO tanqueadas ... (MySQL)
9. TanqueadaService → Retorna TanqueadaDTO con rendimiento calculado
10. TanqueadaController → Response.status(201).entity(dto).build()
11. Jersey → Serializa DTO a JSON
12. ANDROID/WEB ← HTTP 201 + JSON { rendimiento: 42.5 km/l, ... }
```

5.6 Seguridad y Autenticación

- Todos los endpoints protegidos pasan por `JwtFilter`, que valida el Bearer Token en cada solicitud antes de llegar al Controller.
- Las contraseñas se almacenan con hash `bcrypt`. Nunca se guardan en texto plano.
- Los tokens JWT tienen tiempo de expiración configurable (recomendado: 24h para web).

5.7 Patrones de Diseño Aplicados

Patrón	Uso en RodApp	Beneficio
MVC–DAO	Separación Controller – Service – Model	Mantenibilidad y separación clara de responsabilidades
Template Method	GenericDAO como clase base con CRUD reutilizable	Evita código repetido en cada DAO específico
Singleton	JpaUtil.getEntityManagerFactory()	Una sola instancia de EntityManagerFactory en toda la app
Facade	Service envuelve la lógica de múltiples DAOs	Simplifica el acceso desde los Controllers
Filter / Chain of Responsibility	JwtFilter intercepta todas las solicitudes protegidas	Autenticación centralizada, no acoplada al Controller
DTO (Data Transfer Object)	Objetos de respuesta limpios sin exponer entidades JPA	Seguridad y control del JSON retornado al cliente

5.8 Stack de Dependencias Maven

```
<!-- Jakarta EE 10 (API completa) -->
<jakarta.platform:jakarta.jakartaee-api:10.0.0>

<!-- JAX-RS Jersey (implementación REST) -->
<org.glassfish.jersey.core:jersey-server:3.1.1>
<org.glassfish.jersey.media:jersey-media-json-jackson:3.1.1>

<!-- Hibernate JPA (ORM) -->
<org.hibernate:hibernate-core:6.2.0.Final>

<!-- MySQL Connector -->
<com.mysql:mysql-connector-j:8.0.33>

<!-- Seguridad JWT -->
<io.jsonwebtoken:jjwt-api:0.11.5>
<io.jsonwebtoken:jjwt-impl:0.11.5>

<!-- Seguridad Password -->
<org.mindrot:jbcrypt:0.4>
```

```
<!-- Validación Bean -->
<jakarta.validation:jakarta.validation-api:3.0.2>

<!-- Logging -->
<org.slf4j:slf4j-api:2.0.7>
<ch.qos.logback:logback-classic:1.4.11>
```

5.9 Consideraciones para la Fase de Desarrollo

- Paginación: todos los endpoints que retornan listas (tanqueadas, mantenimientos, viajes) deben soportar parámetros page y size para evitar la carga completa de datos en el cliente.
- CORS: configurar filtro CORS para permitir solicitudes desde el origen de la plataforma web durante el desarrollo y producción.
- Variables de entorno: las credenciales de base de datos, el secreto JWT debe manejarse mediante variables de entorno o un archivo de configuración excluido del control de versiones.
- Documentación de API: implementar OpenAPI/Swagger desde el inicio para facilitar la integración entre el equipo de frontend y el equipo de backend.
- Testing: implementar pruebas unitarias con JUnit 5 sobre la capa Service, donde reside la lógica de negocio crítica (cálculo de rendimiento, intervalos de mantenimiento, estado de documentos).